

LESSON 3: SEARCHING LINEAR DATA

One of the fundamental operations on data is searching for a specific item within a dataset. Before we can modify, extract, relocate, or process any data item, we must first locate it accurately. Searching, like all computer operations, is defined and guided by algorithms, each with its own performance characteristics.

In this lesson, we will explore the mechanics and applications of basic search algorithms—linear search and binary search—and compare their performance. Through this comparison, we'll understand how choosing the right algorithm can make a significant difference in efficiency, particularly when dealing with large datasets organized in a linear format.

LINEAR SEARCH

One of the simplest strategies is known as linear search. It is a straightforward approach applied to linear data structures, where the algorithm starts at the first position in the dataset and moves sequentially from one position to the next, checking each data item for a match. If a data item matches the search item, the search ends. Otherwise, it continues until the end of the dataset.

EXAMPLE 3.1

Let's go back to the example of books on a shelf. To search for a particular title using a linear search approach, you would start at the beginning of the shelf and check each book title in order until you find the one you're looking for. However, if you reach the end of the shelf without finding the title, then the book is not on this shelf.

Using a linear search, books on a bookshelf are checked one by one from left to right across the shelf.

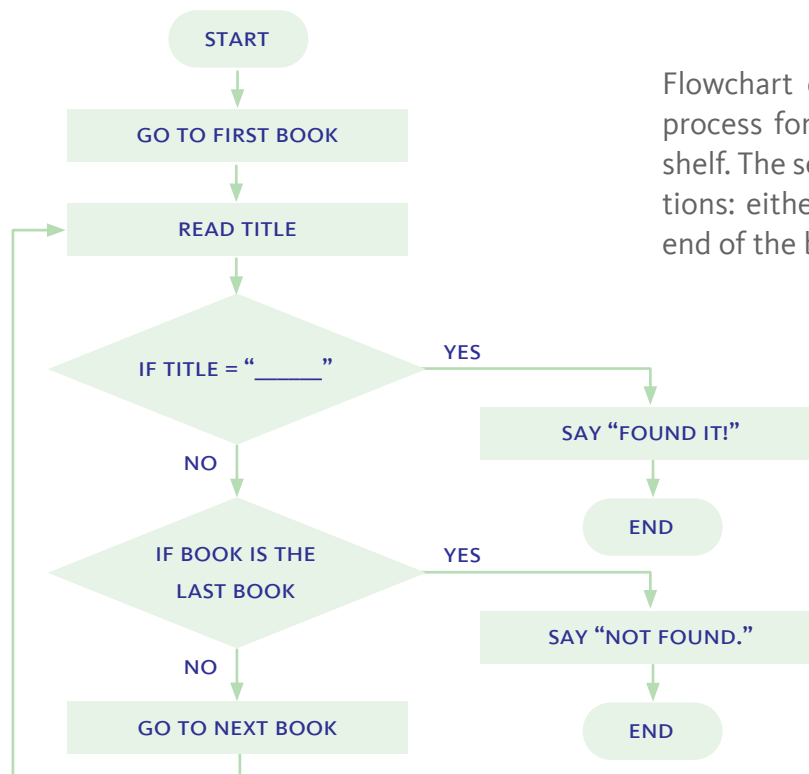


A LINEAR SEARCH ALGORITHM SEQUENTIALLY CHECKS EACH POSITION IN A DATASET.

THE LINEAR SEARCH PROCESS

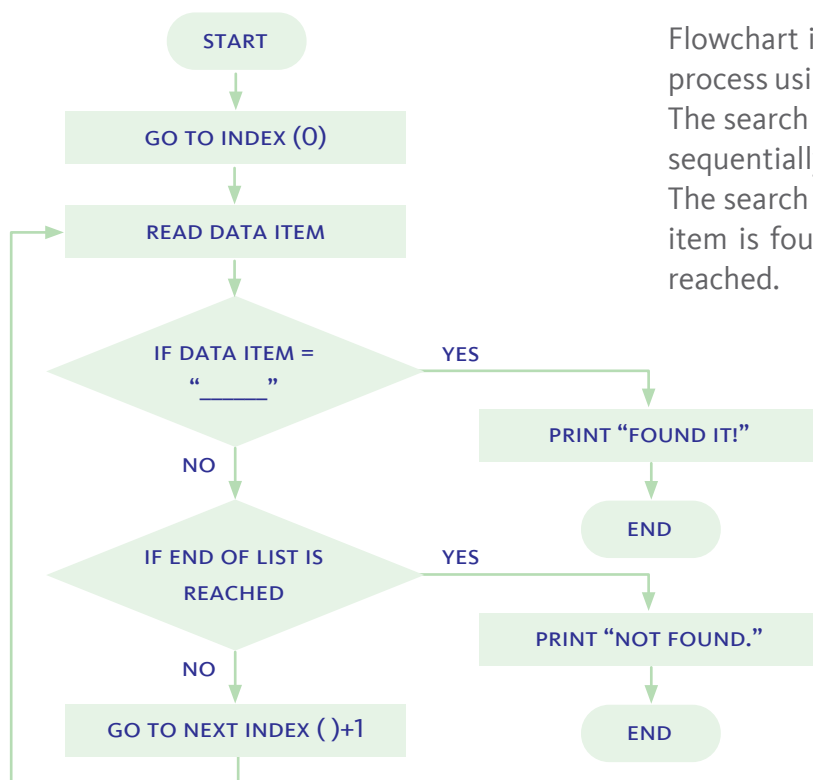
While the process of searching for an item, such as a book on a bookshelf or a toy in a toy basket, may come intuitively, breaking it down into discrete steps—including those required for decision-making—is essential. These steps include deciding to end the search once you’ve found what you’re looking for, or concluding that the item is not on the bookshelf or in the toy basket.

When dealing with any step-by-step process, it is best understood when visually depicted in the form of a flowchart. A well-structured flowchart includes each individual step of the process, a clear flow from one step to the next, and decision points that indicate the path of flow based on each condition that is met.



Flowchart depicting the linear search process for finding a book on a bookshelf. The search ends under two conditions: either the book is found, or the end of the bookshelf is reached.

Now that the basic process of the linear search strategy has been established—even if only for books—we can extrapolate the structure of an algorithm that can be applied to any list of items. Simply described, the algorithm must start at the beginning of the list, sequentially check each item, and end the search if one of two conditions is satisfied: the item is found, or the end of the list is reached.



Flowchart illustrating the linear search process using an index-based approach. The search begins at the first index and sequentially checks each data item. The search ends when either the target item is found, or the end of the list is reached.

Achieving a clear understanding of the process is essential before creating a program that a computer can execute, following the steps of the linear search algorithm. Breaking down the linear search into its fundamental components not only clarifies each action but also prepares us to translate these actions into code.

Next, we will present the linear search algorithm in pseudocode form. This pseudocode provides a step-by-step outline of the process. Each instruction

represents a specific action that the computer will execute, transforming the logical steps of the search process into a precise, executable program.

```
START LinearSearch
OPEN list
SET search_index (0)

REPEAT_UNTIL search_index reaches end of list
  READ data_item AT: search_index

  IF data_item = target THEN
    PRINT "Found it!"
  END LinearSearch
ELSE
  SET search_index (search_index + 1)

PRINT "Not found"
END LinearSearch
```

SET search_index (o): Initializes the starting position of the search to the first index (o), marking the beginning of the list.

REPEAT_UNTIL: Creates a loop that continues until the end of the list is reached. The condition for ending the search upon reaching the last position in the list is embedded within this instruction.

Within the loop, the algorithm **READs** each data item and ends the search **IF** the data item matches the target being searched for.

SET search_index (search_index + 1): Ensures that the search progresses by moving to the next position in the list, incrementing search_index by 1 with each iteration.

The strategy of searching sequentially through a list is straightforward and would work for any list. However, as the size of the list grows, challenges arise. When there are only 10 books on a shelf, checking each one is manageable, but what if the search involves an entire library with hundreds or even thousands of books?

PERFORMANCE

Let's explore the performance of a linear search algorithm. One simple measure of performance is execution time, which refers to the amount of time it takes for the program to complete its run.

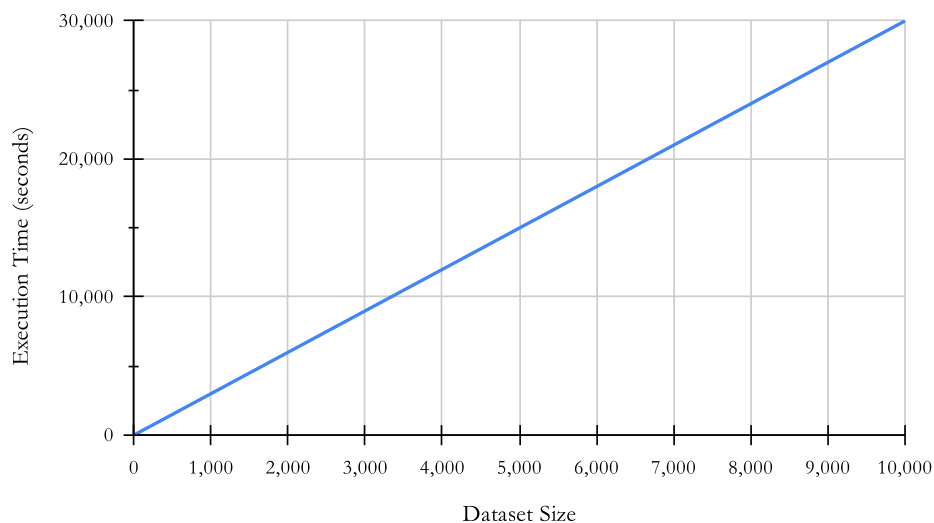
EXAMPLE 3.2

Referring back to the books on the bookshelf, let's assume it takes approximately 3 seconds for a person to check each book, decide whether they've found what they're looking for, or move on to the next book on the shelf. We'll also assume that this person can perform this task tirelessly, without breaks or errors. Since it takes 3 seconds to check each book, the time to search n books would be:

$$\text{Execution Time} = \text{Time per Attempt} \times \text{Number of Attempts}$$

Size of Dataset	Execution Time
10 books	$3 \times 10 = 30 \text{ seconds}$
100 books	$3 \times 100 = 300 \text{ seconds (5 minutes)}$
1,000 books	$3 \times 1,000 = 3,000 \text{ seconds (50 minutes)}$
10,000 books	$3 \times 10,000 = 30,000 \text{ seconds (8 hours 20 minutes)}$

Linear Search Algorithm: Execution Time vs. Dataset Size



The graph illustrates how execution time increases in direct proportion to the size of the dataset when using a linear search algorithm. Each additional book requires a constant amount of time (3 seconds) to check, resulting in a straight-line growth pattern.

WORST-CASE

To best understand the performance of an algorithm, we need to evaluate it in its worst-case scenario. For a search algorithm, this means assuming it has to examine every item in the list before finding the target (or concluding that the target isn't present). This contrasts with a best-case scenario, where the algorithm finds its target on the very first attempt. For a bookshelf, the best case would mean that the target book is the very first one checked.

A WORST-CASE SCENARIO IS THE SITUATION IN WHICH AN ALGORITHM TAKES THE LONGEST POSSIBLE TIME TO ACHIEVE ITS GOAL.

In the previous example, the execution time calculations are based on this worst-case scenario for our linear search, where the algorithm wasn't fortunate enough to find the book among the first few items and had to search all the way to the end of the shelf. This worst-case analysis helps us understand the maximum time our linear search algorithm might require under the least favorable conditions.

Notes

The concepts of algorithm performance in this section are approached with a focus solely on time complexity. This helps students concentrate on understanding how execution time scales with input size before introducing additional factors like space complexity.

Example 3.2 demonstrates that as the dataset size increases, the time required for the search also increases at a constant rate of 3 seconds per item. For instance, searching through 10 books takes about 30 seconds, while searching through 10,000 books takes 30,000 seconds. This reveals a key limitation of the linear search algorithm: for large datasets, the search time becomes impractically long.

EXAMPLE 3.3

Global telephone operators may manage databases with hundreds of millions of telephone numbers. Let's assume one such operator manages 100 million numbers. Further, let's assume its computer systems have a 50-nanosecond latency, meaning it takes the processor about 50 nanoseconds to access memory. A call comes in from a client, and the customer service representative wants to search for this client's account. The execution time for a linear search would be:

$$\text{Execution Time} = 50 \times 100,000,000 = 5,000,000,000 \text{ ns (5 seconds)}$$

A 5-second execution time for a single search in a call center could have significant repercussions for the operator's business. In a call center, where efficiency is crucial, this delay would slow down interactions, leading to longer wait times and potentially frustrating customers.

But the impact extends beyond customer service. Telecom operators rely on real-time systems to route calls, authenticate users, and manage billing. A 5-second delay in verifying or routing each call could disrupt these processes, resulting in dropped calls, misrouted calls, or costly billing errors. Moreover, as the operator's client base grows, so will the delays and related issues, emphasizing the need for more efficient search solutions to maintain smooth, reliable service.

BINARY SEARCH

What if we could use a strategy that reduces the size of the list as the search progresses? The binary search algorithm does exactly that. Instead of starting at the beginning of the list, binary search begins at the middle index. If the target is less than the value of the data item at the middle index, the algorithm knows the target must be in the first half of the list, so it disregards the second half. It then focuses on the middle index of the remaining portion of the list, checking the value there. Based on this comparison, it determines which half of the list to continue searching and which to disregard. This process repeats until the target is found or the list is fully divided, essentially halving the dataset with every iteration.

An important condition for the binary search algorithm to work is that the list must be sorted; otherwise, this strategy would not be effective. While a linear search algorithm works equally well on both sorted and unsorted lists, the binary search algorithm requires a sorted list to function properly. Without sorting, binary search would not be able to accurately determine which half of the list to focus on, making the algorithm ineffective.

A BINARY SEARCH ALGORITHM FINDS A TARGET VALUE IN A SORTED LIST BY REPEATEDLY DIVIDING THE SEARCH RANGE IN HALF.

EXAMPLE 3.4

Let's go back to our books on the bookshelf. There are 12 books on the shelf, and this time, we'll assume the books are sorted in alphabetical order. In a linear search, the worst-case scenario would require the algorithm to check all 12 books. Now, let's see what happens in the worst-case scenario for a binary search algorithm.

The 12 books on the shelf are sorted in alphabetical order.



In order to make the example more tangible, here is a list of all twelve book titles arranged in alphabetical order. In the worst-case scenario, the target book would be at the final possible location after all divisions, requiring multiple attempts to reach it. In this case, our target title is 'Zebra Crossing'. Here's how the binary search would proceed:

```
book_titles = [  
    "A Tale",  
    "Blue Moon",  
    "Crimson Sky",  
    "Deep Waters",  
    "Endless Night",  
    "Frost Fire",  
    "Golden Path",  
    "Hidden World",  
    "Iron Fist",  
    "Jungle King",  
    "Silver Light",  
    "Zebra Crossing"  
]
```

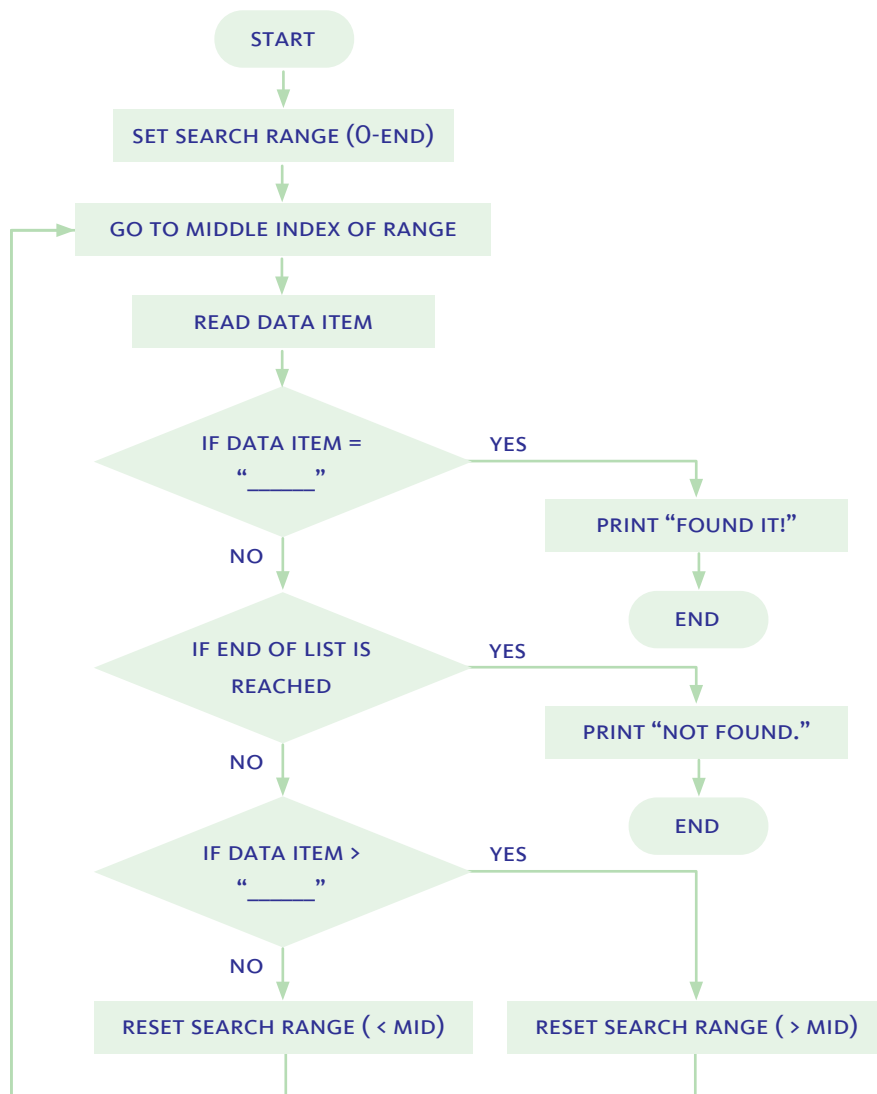
The list of 12 book titles in alphabetical order, which serves as the dataset for the binary search simulation.

The simulation illustrates the steps of a binary search to locate the book title 'Zebra Crossing' within the sorted list. Each attempt shows the search range, middle index, and the decision to disregard half of the list based on comparisons, continuing until the target is found at index 11.

Steps	Range (index)	Middle (index)	Comments
Attempt 1	0 - 11	5	Z > F, disregard 0 - 5
Attempt 2	6 - 11	8	Z > I, disregard 6 - 8
Attempt 3	8 - 11	10	Z > S, disregard 8 - 10
Attempt 4	11	11	Found it!

Had this search been conducted using a linear search algorithm, it would have taken 12 attempts. In comparison, the binary search algorithm's worst-case scenario required only 4 attempts, demonstrating the performance advantage of binary search over linear search when working with sorted lists.

Now that the basic concept of a binary search algorithm has been introduced, we can represent it in a flowchart that illustrates its essential steps: 1) setting the search range, 2) identifying the middle index of the search range, 3) evaluating the data item at the middle index to determine if the target has been found, and 4) if the target is not found, defining the next range of indices to continue the search.



This flowchart illustrates the steps of a binary search algorithm. Starting with an initial search range, the algorithm identifies the middle index, evaluates the data item at that index, and determines if it matches the target. If the target is not found, the algorithm narrows the search range by selecting the appropriate half and repeats the process until the target is located or the search range is exhausted.

Before writing the algorithm in pseudocode, we should explore its fundamental aspects. The first is setting the search range and identifying the search index, and the second is understanding the main loop of the algorithm and the conditions that repeat within it.

THE SEARCH RANGE AND INDEX

The search range in a binary search algorithm is always confined between its lower end, the **start_index**, and its upper end, the **end_index**. When the algorithm begins, these indices are set to the first and last positions of the list: **start_index (0)** and **end_index (last index of the dataset)**.

Once the search range has been set, the next step is to calculate the midpoint of the range to identify the index at which to search, the **search_index**. This midpoint is determined by halving the sum of the **start_index** and **end_index** of the search range.

$$\text{search_index} = \text{midpoint} = (\text{start_index} + \text{end_index}) / 2$$

With each iteration of the loop, the search range is redefined by adjusting either the upper end, **end_index**, or the lower end, **start_index**, depending on which half of the current range the target is determined to be in.

- If the target is in the lower half of the current range, **end_index** is adjusted to one position before the middle, calculated as **search_index - 1**.
- If the target is in the upper half of the current range, **start_index** is adjusted to one position after the middle, calculated as **search_index + 1**.

This approach continually narrows down the search range, recalculating the midpoint with each iteration that resets the range.

Notes

The midpoint must be an integer. If the calculation results in a fractional number, round down to the nearest integer.

THE LOOP AND CONDITIONS

The loop in the binary search algorithm ensures that a specific set of steps is repeated for each search index and that certain conditions are evaluated:

- Has the target been found at the search index: Is **data_item = target**?
- Is the target in the lower half of the search range: Is **data_item > target**?
- Is the target in the upper half of the search range: Is **data_item < target**?
In this algorithm, if **data_item** is neither equal to nor greater than the target, then it must be less than the target. This condition is therefore implied rather than explicitly stated.
- Has the whole list been searched? This is embedded within the **REPEAT_UNTIL** loop, which ensures that the loop continues until a specific condition is met—in this case, when the **search_index** has reached the end of the list.

```
START BinarySearch
OPEN list
SET start_index (0)
SET end_index (last index of list)

REPEAT_UNTIL search_index reaches end of list
  SET midpoint = (start_index + end_index)/2
  SET search_index (midpoint)
  READ data_item AT: search_index

  IF data_item = target THEN
    PRINT "Found it!"
    END BinarySearch
  ELSE IF data_item > target THEN
    SET end_index (search_index - 1)
  ELSE
    SET start_index (search_index + 1)

PRINT "Not found"
END BinarySearch
```

Pseudocode for a binary search algorithm that repeatedly narrows down the search range by adjusting the start and end indices until the target is found or the list is fully searched.

PERFORMANCE

Previously, in Example 3.4, we compared the performance of linear and binary search algorithms by counting the attempts required to find a target. In this worst-case scenario of searching through 12 book titles, the binary search algorithm located its target in significantly fewer attempts than the linear search—4 attempts versus 12.

We can also compare the two algorithms based on their execution time. For a linear search algorithm, the execution time is the product of the time required for a single attempt and the total number of attempts needed to identify the target. In contrast, while the execution time for a binary search algorithm also depends on the time required for each attempt, the number of possible attempts is halved with each step. To represent this reduction mathematically, we use the log-base 2 operation, which captures the exponential decrease in search range for each iteration.

If t is the time it takes to execute one attempt, and n is the number of items in the dataset, then the execution time for a binary search is:

$$\text{Execution Time} = t \times \log_2(n)$$

EXAMPLE 3.5

Referring back to the books on the shelf in Example 3.2, we will now use a binary search algorithm instead of a linear search. We'll compare the execution times for datasets of the same sizes to highlight the efficiency of binary search. The assumption that it takes 3 seconds to check each book will still hold.

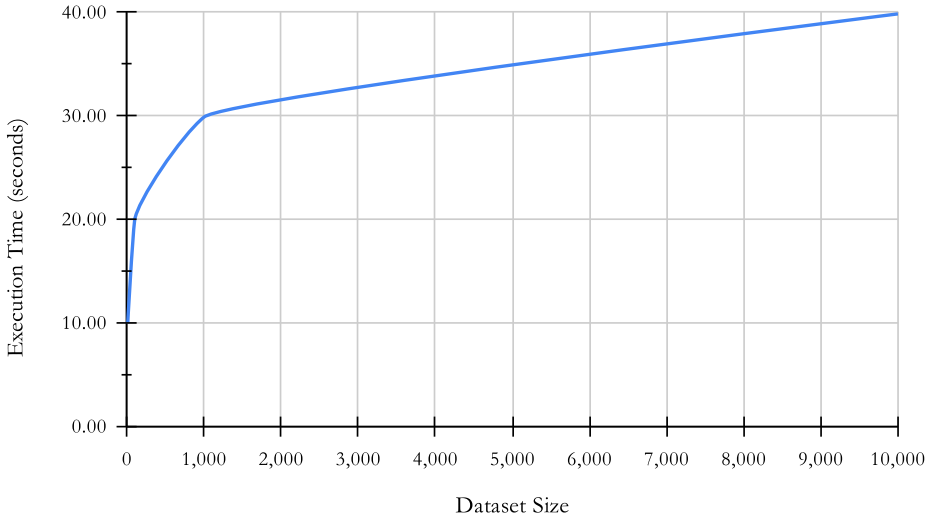
Notes

Students are not expected to understand the logarithmic function in depth at this stage. However, they should grasp that the log-base 2 function is used to mathematically represent the exponential decrease in the search range with each iteration of the binary search algorithm.

Size of Dataset	Execution Time (Binary Search)	Execution Time (Linear Search)
10 books	$3 \times \log_2(10) = 9.97 \text{ seconds}$	30 seconds
100 books	$3 \times \log_2(100) = 19.93 \text{ seconds}$	300 seconds (5 minutes)
1,000 books	$3 \times \log_2(1,000) = 29.90 \text{ seconds}$	3,000 seconds (50 minutes)
10,000 books	$3 \times \log_2(10,000) = 39.86 \text{ seconds}$	30,000 seconds (8 hours 20 minutes)

The table highlights a drastic difference in performance between linear search and binary search algorithms. For a dataset of 10,000 books, the execution time for a linear search grows linearly, reaching 8 hours and 20 minutes, while the execution time for a binary search grows logarithmically, reaching only 39.86 seconds. This substantial difference arises because binary search halves the search range with each attempt, making it far more efficient for larger datasets.

Binary Search Algorithm: Execution Time vs. Dataset Size



Execution time of a binary search algorithm across different dataset sizes, illustrating logarithmic growth and the algorithm’s efficiency as dataset size increases.

EXAMPLE 3.6

If the global telephone operator in Example 3.3 used a binary search algorithm instead of a linear search to locate a telephone number in a database of 100 million numbers, with a computer system latency of 50 nanoseconds per attempt, the execution time for the binary search would be:

$$\text{Execution Time} = 50 \times \log_2(100,000,000) = 1,330 \text{ nanoseconds}$$

Compared to the 5 seconds required for a linear search to accomplish this task, the 1,330 nanoseconds (1.33 microseconds) for a binary search is a significant improvement in performance.

CONCLUSION

In this lesson, we explored two fundamental search algorithms—linear search and binary search—and compared their performance across varying dataset sizes. While linear search is simple and works on unsorted data, its execution time grows linearly, making it inefficient for large datasets. In contrast, binary search, which requires sorted data, drastically reduces the search range with each attempt, resulting in logarithmic growth in execution time. Understanding these differences highlights the importance of choosing the right algorithm based on dataset size and performance requirements, a key concept in computer science.

SUMMARY

Searching is a fundamental operation needed to locate specific items in a dataset before performing further actions like modification, extraction, or processing.

Linear Search:

- Works by sequentially checking each item in the dataset from start to finish.
- Can be used on both sorted and unsorted datasets.
- Execution time grows linearly with the size of the dataset, making it inefficient for large datasets.

Binary Search:

- Requires the dataset to be sorted.
- Starts by checking the middle item of the search range and eliminates half of the range with each attempt.
- Execution time grows logarithmically with the size of the dataset, making it much more efficient than linear search for large datasets.

Comparison of Performance:

- Linear search checks every item, which results in a high execution time as the dataset grows.
- Binary search dramatically reduces the number of attempts by halving the search range, resulting in faster execution times for large, sorted datasets.
- Understanding the performance differences helps in choosing the right algorithm based on dataset size and structure.

Activity 3.1

Read the following statements, and write on the line provided for each statement 'T' if it is true and 'F' if it is false.

T/F	STATEMENT
<u>T</u>	A linear search algorithm can be used on both sorted and unsorted datasets.
<u>F</u>	A binary search algorithm can be used on both sorted and unsorted datasets.
<u>F</u>	A binary search algorithm begins by checking the first item in the dataset.
<u>F</u>	In a linear search algorithm, the search range is halved with each iteration.
<u>T</u>	Execution time for a linear search increases directly with the size of the dataset.
<u>T</u>	Execution time for a binary search increases logarithmically with the size of the dataset.
<u>F</u>	In the worst-case scenario, binary search requires more attempts than linear search.
<u>T</u>	In the best-case scenario, binary search and linear search are equally efficient.
<u>T</u>	In the worst-case scenario, binary search is more efficient than linear search, regardless of dataset size.
<u>F</u>	The choice of search algorithm does not impact the overall performance of a program.

Activity 3.2

Answer the questions below.

1. What is the primary characteristic of a linear search algorithm?
 - a) It requires a sorted dataset.
 - b) **It searches each item sequentially.**
 - c) It divides the dataset in half each time.
 - d) It starts at the middle of the dataset.
2. Which of the following is a limitation of linear search for large datasets?
 - a) It requires a sorted list.
 - b) **It can be very slow because it checks each item sequentially.**
 - c) It cannot be used with unique values.
 - d) It does not work on datasets with numbers.
3. What is the best-case scenario for a linear search algorithm?
 - a) The target is in the middle of the dataset.
 - b) The target is at the end of the dataset.
 - c) The dataset is already sorted.
 - d) **The target is the first item in the dataset.**
4. Which type of data structure can linear search be applied to?
 - a) Only sorted arrays
 - b) Only linked lists
 - c) **Any list, sorted or unsorted**
 - d) Only binary trees
5. What happens in a linear search if the target item is not in the dataset?
 - a) The search stops at the midpoint of the dataset.
 - b) **The search checks every item and then stops.**
 - c) The search requires the dataset to be reordered.
 - d) The search skips some items to save time.

Activity 3.3

Answer the questions below.

1. What is the main requirement for using a binary search algorithm on a dataset?
 - a) The dataset must contain unique values.
 - b) **The dataset must be sorted.**
 - c) The dataset must be large.
 - d) The dataset must be stored in a linked list.
2. What does binary search do with each iteration?
 - a) Moves one step closer to the target from the beginning of the list.
 - b) **Divides the search range in half.**
 - c) Checks each item sequentially.
 - d) Moves in reverse through the list.
3. In a binary search, if the target is less than the middle item, what should the algorithm do next?
 - a) **Focus on the lower half of the search range.**
 - b) Focus on the upper half of the search range.
 - c) Ignore the entire list.
 - d) Conclude that the target is not present.
4. What is the best-case scenario for a binary search algorithm?
 - a) The dataset is very large.
 - b) The dataset is unsorted.
 - c) The target is the first item in the dataset.
 - d) **The target is the middle item on the first check.**
5. What is the growth rate of a binary search algorithm?
 - a) Linear
 - b) Exponential
 - c) **Logarithmic**
 - d) Quadratic

Activity 3.4

Answer the questions below.

1. Which search algorithm is generally faster for large, sorted datasets?
 - a) **Binary search**
 - b) Linear search
 - c) Both are equally fast
 - d) It depends on the dataset size
2. What is a key difference in requirements between linear and binary search?
 - a) Linear search requires sorted data, while binary search does not.
 - b) **Binary search requires sorted data, while linear search does not.**
 - c) Both require sorted data.
 - d) Neither requires sorted data.
3. When would a linear search be preferred over a binary search?
 - a) When there are fewer than 10 items.
 - b) When the dataset is sorted and very large.
 - c) When binary search fails to find the target.
 - d) **When the dataset is unsorted.**
4. Why is binary search more efficient than linear search on large, sorted datasets?
 - a) It skips every other item in the list.
 - b) It checks each item twice.
 - c) **It divides the search range in half with each attempt.**
 - d) It starts at the end of the list.
5. If the target is the first item in a dataset, how does the performance of binary search compare to linear search?
 - a) Binary search will be slower.
 - b) Linear search will be slower.
 - c) **Both will perform the same.**
 - d) The result depends on dataset size.

Activity 3.5

Refer to the sorted list. Compare the number of attempts required for linear search and binary search to find specific numbers in the list.

List_A = [3, 7, 12, 19, 25, 31, 44, 52, 67, 72, 88, 93, 105, 117, 126, 139, 145, 159, 167, 178, 190]

- Find the number 3.

Linear Search Attempts:

1

Binary Search Attempts:

5

- Find the number 88.

Linear Search Attempts:

11

Binary Search Attempts:

1

- Find the number 190.

Linear Search Attempts:

21

Binary Search Attempts:

5

- Find the number 105.

Linear Search Attempts:

13

Binary Search Attempts:

3

- What is the number of worst-case scenario attempts?

Linear Search Attempts:

21

Binary Search Attempts:

5

- Which index in the list provides a best-case scenario for:

Linear Search:

index (0) first

Binary Search:

index (10) middle

Activity 3.6

For each of the three scenarios provided below, determine the number of attempts that a linear and a binary search algorithm would require to find the target item in the worst-case scenario. Then, calculate the execution time for each algorithm's search.

1. A dataset contains 1,024 items, and each attempt takes 2 milliseconds. Calculate the worst-case number of attempts and the execution time for both a linear and a binary search on this dataset.

Linear Search: Attempts: $n = 1,024$ (all items are checked)

Execution time = $t \times n = 2 \times 1,024 = 2,048$ ms (about 2 seconds)

Binary Search: Attempts: $\log_2(n) = \log_2(1,024) = 10$

Execution time = $t \times \log_2(n) = 2 \times 10 = 20$ ms

2. A dataset of 20,000 items is searched, with each attempt taking 5 milliseconds. Calculate the worst-case number of attempts and the execution time for both a linear and a binary search.

Linear Search: Attempts: $n = 20,000$ (all items are checked)

Execution time = $t \times n = 5 \times 20,000 = 100,000$ ms (100 seconds)

Binary Search: Attempts: $\log_2(n) = \log_2(20,000) = 14.29 = 15$ (round up)

Execution time = $t \times \log_2(n) = 5 \times 15 = 75$ ms

3. A large dataset of 1,000,000 items is searched, and each attempt takes 10 milliseconds. Calculate the worst-case number of attempts and the execution time for both a linear and a binary search.

Linear Search: Attempts: $n = 1,000,000$ (all items are checked)

Execution time = $t \times n = 10 \times 1,000,000 = 10,000,000$ ms (10,000 seconds)

Binary Search: Attempts: $\log_2(n) = \log_2(1,000,000) = 19.93 = 20$ (round up)

Execution time = $t \times \log_2(n) = 10 \times 20 = 200$ ms

Activity 3.7

Use the linear search algorithm provided in pseudocode to answer the following questions.

```

START LinearSearch
OPEN list
SET search_index (0)

REPEAT_UNTIL search_index reaches end of list
  READ data_item AT: search_index

  IF data_item = target THEN
    PRINT "Found it!"
    END LinearSearch
  ELSE
    SET search_index (search_index + 1)

PRINT "Not found"
END LinearSearch

```

1. What is the purpose of the instruction: SET search_index (0) ?

Sets the starting position of the search at the first item in the list.

2. Which instruction increments the search index by 1 position?

SET search_index (search_index + 1)

3. What is the purpose of the instruction: REPEAT_UNTIL ?

It creates a loop to repeatedly check items in the list.

4. Which two conditions end the REPEAT_UNTIL loop?

search_index reaches end of list

data_item = target

Activity 3.8

Use the binary search algorithm provided in pseudocode to answer the following questions.

```

START BinarySearch
OPEN list
SET start_index (0)
SET end_index (last index of list)

REPEAT_UNTIL search_index reaches end of list
  SET midpoint = (start_index + end_index)/2
  SET search_index (midpoint)
  READ data_item AT: search_index

  IF data_item = target THEN
    PRINT "Found it!"
    END BinarySearch
  ELSE IF data_item > target THEN
    SET end_index (search_index - 1)
  ELSE
    SET start_index (search_index + 1)

PRINT "Not found"
END BinarySearch

```

1. Which two parameters define the search range?

The two parameters are the start_index and the end_index.

2. What is the purpose of the midpoint variable?

The midpoint variable calculates the middle index of the current search range.

3. What is the purpose of the instruction: SET end_index (search_index - 1) ?

It focuses the search on the lower half of the current search range by resetting the current end index to one position below the current midpoint.

4. Why is the instruction IF data_item < target THEN not included?

If data_item is neither equal to nor greater than the target, then it must be less than the target by exclusion.

Activity 3.9

Having gained an in-depth understanding of the mechanics, performance, and logic behind linear and binary search algorithms, it's time to explore them further. This activity allows students to examine linear and binary search algorithms in Python programs. Follow the links to download the three programs provided for this activity.

1. [Linear Search Program](#): This program performs a linear search on a user-defined, sorted range. It checks each item in the list sequentially until it finds the target or reaches the end. The program outputs the number of attempts taken, demonstrating the step-by-step process of linear search.
2. [Binary Search Program](#): This program performs a binary search on a user-defined, sorted range. It begins by checking the midpoint of the range and repeatedly halves the search space based on comparisons. The program reports the number of attempts, showing the efficiency of binary search on sorted data.
3. [Combined Search Program](#): This program includes both linear and binary search algorithms, operating on the same user-defined, sorted range. It allows users to compare the performance of linear and binary search directly by running both methods on the same target and range. This comparison highlights the efficiency differences between the two search methods.

Notes

Notice the similarity between the pseudocode and the Python programs provided for linear and binary search. This similarity highlights how pseudocode serves as an essential bridge between algorithmic logic and actual coding. Pseudocode provides students with a clear, language-agnostic structure for algorithms, helping them transition smoothly from planning to implementation in Python or other programming languages. Encourage students to use pseudocode as a guide to understand, develop, and refine their programming skills.

